#5 Also create another hash table to keep track, for each vertex `u`, of the vertex through which `u` was reached. This dictionary can later be used to reconstruct the path from `start` to the goal.
#6 Initialize vertex distances to infinity (or, equivalently, to the largest value that can be stored).
#7 For the starting point only, we need to set its distance (from itself) to 0.
#8 Start a loop, running until the queue is empty. It will run at least once, because of line #3.
#9 Dequeue the head of the queue (equivalently, extract the top of a priority queue). This will become the current vertex.
#10 If we reached the goal, we are done, just return current vertex. The function `isGoal` can abstract condition checked to find the goal vertices: it can be reaching one or more specific goal vertices, or getting to vertices that satisfy a certain condition.
#11 Iterate over the outgoing edges for current vertex.
#12 If this vertex hasn't been discovered yet (its distance has never been set) then the path with fewer edges from `start` to `u` certainly passes through `v` (because of the way we are expanding the search frontier).
#13 Consequently, the first time a vertex is discovered, set its distance, its parent, and then add it to the queue, so it can later be visited in a future iteration.
#14 If the goal is never reached, we need to return `null` or, equivalently, another value that signals that search failed.

At its core, this algorithm keeps a queue of vertices that will be visited next, the so-called *frontier*; vertices are kept in a specific order, so that they are processed from the closest